

INDIA DATA RESEARCH ACADEMY
IDRA | CAPSTONE PROJECT

CUSTOMER CHURN PREDICTION IN A TELECOMMUNICATIONS COMPANY

Project Report Submitted in fulfillment of the Requirements for the Award of the Internship of Summer
Training Program by IDRA

Data Science and AI

By:

Name: SRIRAM R

Institute: KONGU ENGINEERING COLLEGE

Institute Roll No: 24CSR304

Enrollment No.: IDRA-2026-755945

Under the Supervision of

Dr. Shaheena Salam

Through the platform:

India Data Research Academy

DECLARATION

I hereby declare that the work presented in this capstone project report, titled “Customer Churn Prediction in a Telecommunications Company,” has been prepared as part of the IDRA Data Science and AI training program. The analysis, preprocessing workflow, exploratory analysis, statistical analysis, and machine learning implementation are based on the supplied project code and dataset workflow. All external sources used in the final version of the report should be acknowledged appropriately.

ACKNOWLEDGEMENT

I express my sincere gratitude to the India Data Research Academy (IDRA) for providing the training platform and capstone framework for this project. I thank Dr. Shaheena Salam for her supervision and guidance. I also acknowledge the source of the Telco Customer Churn dataset and the open-source Python libraries used to implement the analysis. Finally, I thank everyone who provided academic and technical support during the completion of this work.

ABSTRACT

Customer churn is an important business problem for telecommunications companies because losing existing customers can reduce recurring revenue and increase the effort required to acquire replacement customers. This project develops a data-driven churn prediction workflow using the Telco Customer Churn dataset. The implementation loads the customer records, converts the TotalCharges variable to a numeric format, handles missing TotalCharges values using median imputation, removes the customerID identifier from the predictive feature set, and encodes the target variable Churn as a binary outcome. Exploratory analysis focuses on churn distribution, contract type, and monthly charges. A chi-square test is used to examine the association between contract type and churn. For prediction, a Logistic Regression classifier is implemented within a preprocessing pipeline. Numerical variables are median-imputed and standardized, while categorical variables are imputed using the most frequent category and one-hot encoded. The dataset is divided into 80% training and 20% testing data using stratification, and the model uses class balancing to address the unequal class distribution. Evaluation is based on accuracy, precision, recall, F1-score, ROC AUC, and the confusion matrix. The supplied implementation also extracts model coefficients to identify influential encoded features. The resulting workflow provides a reproducible foundation for identifying customers who may be at higher risk of churn and for supporting targeted customer-retention strategies.

Keywords: Customer Churn; Telecom Analytics; Logistic Regression; Classification; Predictive Analytics

TABLE OF CONTENTS

1. 1. Introduction
2. 2. Problem Statement and Motivation
3. 3. Objectives
4. 4. Research Questions
5. 5. Dataset Description and Data Understanding
6. 6. Data Cleaning
7. 7. Data Preprocessing
8. 8. Exploratory Data Analysis
9. 9. Statistical Analysis
10. 10. Feature Engineering
11. 11. Model Development
12. 12. Model Evaluation
13. 13. Results and Findings
14. 14. Discussion
15. 15. Limitations
16. 16. Conclusion
17. 17. Recommendations and Future Work
18. 18. References
19. Appendix A. Code
20. Appendix B. Additional Figures and Tables

Note: Update the Word Table of Contents after final formatting if automatic page numbers are required.

1. INTRODUCTION

1.1 Background

Customer churn refers to a customer discontinuing a service relationship with a company. In telecommunications, churn can affect recurring revenue and can require additional marketing and acquisition activity to replace customers who leave. Because telecom providers collect information about customer tenure, contract arrangements, service subscriptions, billing, and charges, these data can be analyzed to identify patterns associated with churn.

1.2 Context of the Problem

The project is designed around a telecom customer dataset in which Churn is the outcome to be predicted. The practical stakeholders are telecom business and customer-retention teams that may use churn-risk information to identify customers for proactive engagement. The supplied implementation examines contract type and monthly charges during exploratory analysis and uses a Logistic Regression model to generate a binary churn prediction.

1.3 Scope of the Study

The study covers data cleaning, preprocessing, exploratory analysis, a statistical association test, supervised classification, model evaluation, and interpretation of Logistic Regression coefficients. It does not cover live deployment, real-time customer scoring, causal inference, experimental retention campaigns, or financial estimation of customer lifetime value. The conclusions are limited to the supplied dataset and implemented workflow.

2. PROBLEM STATEMENT AND MOTIVATION

The problem addressed in this project is: “Can customer-level information from a telecommunications dataset be used to predict whether a customer is likely to churn, and which observed characteristics are associated with the churn outcome?”

The motivation is to transform historical customer information into an analytical signal that can support retention planning. A predictive classification model can help prioritize customers for further business review, while exploratory and statistical analysis can reveal relationships that are useful for understanding churn patterns.

3. OBJECTIVES

- Prepare the Telco Customer Churn dataset for reliable analysis by correcting data types and handling missing values.
- Identify important patterns in churn with respect to customer contract arrangements and monthly charges.
- Examine the statistical association between contract type and churn.
- Build a Logistic Regression classification model to predict the binary Churn outcome.
- Evaluate the model using accuracy, precision, recall, F1-score, ROC AUC, and a confusion matrix.
- Interpret model coefficients and translate the analytical results into practical customer-retention recommendations.

4. RESEARCH QUESTIONS

21. What are the main characteristics and data-quality issues in the Telco Customer Churn dataset?
22. How is churn distributed across the customer population?

23. How does churn vary across contract types and monthly-charge levels?
24. Is there a statistically significant association between contract type and churn?
25. How accurately can Logistic Regression classify customers as churners or non-churners?
26. Which encoded features have the strongest Logistic Regression coefficients?

5. DATASET DESCRIPTION AND DATA UNDERSTANDING

5.1 Data Source

The project code loads a CSV file named “P_4_WA_Fn-UseC_-Telco-Customer-Churn.csv”. The exact external dataset URL, access date, and licensing statement are not specified in the supplied Python file; therefore these details should be inserted from the dataset source actually used for submission.

5.2 Dataset Size and Structure

The standard Telco Customer Churn dataset represented by the supplied workflow contains 7,043 customer records and 21 original variables. The code adds one derived variable, Churn_Code, for modelling, resulting in 22 columns after the cleaning step. The prediction target is Churn, which is converted into Churn_Code where Yes = 1 and No = 0.

Item	Value
Original records	7,043
Original variables	21
Derived modelling variable	Churn_Code
Post-cleaning columns	22
Target	Churn / Churn_Code
Identifier removed for modelling	customerID
Problem type	Binary classification

Table 1. Summary of the Dataset

The supplied code calculates summary statistics including average tenure, average monthly charges, average total charges, churn rate, and churn rate by contract type. These values are intended to be reported from the executed project output in the final submission.

5.3 Data Dictionary

Variable	Meaning	Type	Role
customerID	Unique customer identifier	Categorical/Text	Identifier
gender	Customer gender category	Categorical	Feature
SeniorCitizen	Senior-citizen indicator	Numeric/Binary	Feature
Partner	Whether customer has a partner	Categorical	Feature
Dependents	Whether customer has dependents	Categorical	Feature
tenure	Number of months with the company	Numeric	Feature
PhoneService	Phone service subscription	Categorical	Feature
MultipleLines	Multiple-line service category	Categorical	Feature
InternetService	Internet service category	Categorical	Feature
OnlineSecurity	Online security subscription	Categorical	Feature
OnlineBackup	Online backup subscription	Categorical	Feature
DeviceProtection	Device protection subscription	Categorical	Feature
TechSupport	Technical support subscription	Categorical	Feature
StreamingTV	Streaming TV subscription	Categorical	Feature
StreamingMovies	Streaming movies subscription	Categorical	Feature
Contract	Contract type	Categorical	Feature
PaperlessBilling	Paperless-billing indicator	Categorical	Feature

PaymentMethod	Payment method	Categorical	Feature
MonthlyCharges	Current monthly charge	Numeric	Feature
TotalCharges	Total charges recorded for customer	Numeric	Feature
Churn	Customer churn outcome	Categorical	Target
Churn_Code	Binary encoding of Churn	Numeric	Target used by model

Table 2. Data Dictionary

6. DATA CLEANING

The supplied implementation performs the core cleaning operations before modelling. TotalCharges is explicitly converted from text to numeric using coercion; values that cannot be converted become missing. The Churn text values are stripped of surrounding whitespace. Missing TotalCharges values are then replaced using the median of TotalCharges. Finally, Churn_Code is created from the mapping Yes = 1 and No = 0.

6.1 Missing Values

The code specifically handles missing or non-numeric TotalCharges values by converting invalid entries to missing values and filling them with the median. The exact missing-value count before imputation is not printed by the supplied script and should be taken from the executed dataset inspection if the final report requires the count.

6.2 Duplicate Records

The supplied Python implementation does not explicitly calculate or remove duplicate rows. Therefore, duplicate-record status should not be claimed as verified from the code alone.

6.3 Data Types and Inconsistent Values

TotalCharges is converted to numeric form using pandas.to_numeric with errors='coerce'. Churn values are normalized by applying str.strip(). These steps make the target and billing variable suitable for downstream processing.

6.4 Outlier Handling

The code visualizes MonthlyCharges using a boxplot, but it does not remove or winsorize outliers. Therefore, observations are retained rather than being removed without justification.

6.5 Before vs After Cleaning

Aspect	Before Cleaning	Cleaning Action	After Cleaning
TotalCharges	May be stored as text	Convert to numeric; invalid values become missing	Numeric variable
Missing TotalCharges	Present where conversion fails	Median imputation	No missing values from this step
Churn	Text categories	Strip whitespace	Normalized Yes/No
Target encoding	Churn text only	Map Yes→1 and No→0	Churn_Code created
customerID	Customer identifier	Excluded from model features	Not used for prediction

Table 3. Data Quality Before and After Cleaning

7. DATA PREPROCESSING

The preprocessing workflow is implemented as a scikit-learn ColumnTransformer inside a Pipeline. This is important because the preprocessing transformations are fitted using the training data and then applied consistently to the test data, reducing the risk of data leakage.

7.1 Categorical Encoding

Categorical variables are imputed using the most frequent value and then transformed using OneHotEncoder(handle_unknown='ignore'). One-hot encoding converts category values into numerical indicator variables while allowing previously unseen categories in the test or future data to be handled without an encoding error.

7.2 Feature Scaling and Transformation

Numerical variables are median-imputed and standardized using StandardScaler. Scaling is appropriate for Logistic Regression because it places numerical variables on comparable scales and supports stable coefficient estimation.

7.3 Train-Test Split

The code uses an 80:20 train-test split with random_state=42 and stratify=y. Stratification preserves approximately the same class proportion in the training and test sets. The preprocessing object is included in the modelling pipeline, so it is fitted as part of the training process rather than using test-set statistics.

7.4 Final Feature Set

The model uses all columns except customerID, Churn, and Churn_Code as input features. Churn_Code is the binary target. Thus, the input set consists of the remaining customer demographic, service, contract, billing, tenure, and charge variables.

8. EXPLORATORY DATA ANALYSIS

The supplied code creates three principal EDA visualizations: customer churn distribution, churn by contract type, and monthly charges by churn status.

8.1 Univariate Analysis

The churn distribution plot is designed to show the number of customers in each Churn category. The standard dataset represented by the project has a minority churn class, with approximately 26.5% of customers marked as churned. This class imbalance is relevant to model evaluation because accuracy alone can hide poor performance on the minority class.

[INSERT FIGURE 1 HERE — Customer Churn Distribution Plot generated by the supplied Python code]

Figure 1. Customer Churn Distribution

8.2 Categorical Analysis

The contract analysis uses a stacked bar chart to compare churn counts across Month-to-month, One year, and Two year contract categories. The project code and accompanying report logic identify month-to-month customers as the group with substantially higher churn.

[INSERT FIGURE 2 HERE — Churn by Contract Type generated by the supplied Python code]

Figure 2. Churn by Contract Type

8.3 Bivariate / Relationship Analysis

The MonthlyCharges boxplot compares the distribution of monthly charges between churned and non-churned customers. The supplied report logic notes that monthly charges are noticeably higher among customers who churn. This is an observed association and should not be interpreted as proof that higher charges cause churn.

[INSERT FIGURE 3 HERE — Monthly Charges by Churn Status generated by the supplied Python code]

Figure 3. Monthly Charges by Churn Status

8.4 Correlation Analysis

The supplied Python implementation does not create a numerical correlation heatmap. Therefore, no correlation matrix is reported here as if it had been calculated by the submitted code.

8.5 Other Figures

Additional figures can be placed in Appendix B after executing the project and saving the generated plots.

9. STATISTICAL ANALYSIS

The project performs a chi-square test of independence between Contract and Churn. This test is appropriate for examining whether the observed distribution of churn differs across categorical contract groups.

The exact chi-square statistic and p-value are generated at runtime by the supplied Python program. They should be copied into the final version after execution rather than being estimated here.

Statistical Test	Variables	Statistic	p-value	Interpretation
Chi-square test of independence	Contract × Churn	[Run code]	[Run code]	Use $p < 0.05$ as evidence of statistical association; do not interpret this as causation.

Table 4. Statistical Test Result

The code is written to compare the p-value with the 0.05 significance level and, in the generated PDF narrative, treats a very small p-value as evidence that contract type and churn are statistically associated.

10. FEATURE ENGINEERING

The project uses limited explicit feature engineering. The main derived target variable is Churn_Code, created by mapping Churn = Yes to 1 and Churn = No to 0. The modelling pipeline also creates one-hot encoded variables from categorical columns and standardized numerical variables. These transformations are modelling representations rather than new business variables.

Feature / Transformation	Logic	Purpose
Churn_Code	Yes → 1; No → 0	Binary target for classification
One-hot encoded categorical features	One indicator per category	Convert categorical inputs into numeric model features
Standardized numerical features	StandardScaler applied to numeric inputs	Put numerical predictors on a comparable scale

Table 5. Engineered Features and Transformations

11. MODEL DEVELOPMENT

11.1 Problem Formulation

This is a binary classification problem. The input consists of customer demographic, tenure, service, contract, billing, and charge variables, while the target is Churn_Code, where 1 represents churn and 0 represents no churn.

11.2 Model Selection

Logistic Regression was selected because the project requires a binary classification model and the algorithm provides interpretable coefficients. The supplied implementation additionally uses `class_weight='balanced'`, which adjusts the learning objective to account for the unequal class distribution.

11.3 Training the Model

The model is implemented as a single pipeline containing the ColumnTransformer preprocessing stage followed by LogisticRegression(`max_iter=5000`, `class_weight='balanced'`, `random_state=42`). The model is trained on the 80% training partition and evaluated on the 20% held-out test partition.

Key implementation excerpt:

```
X = df.drop(columns=['customerID', 'Churn', 'Churn_Code'])
y = df['Churn_Code']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

model = LogisticRegression(
    max_iter=5000, class_weight='balanced', random_state=42
)
```

12. MODEL EVALUATION

The supplied implementation evaluates the classifier using Accuracy, Precision, Recall, F1 Score, ROC AUC, and the confusion matrix. These measures provide complementary information: accuracy summarizes overall correctness, precision measures the proportion of predicted churners that are actually churners, recall measures how many actual churners are detected, F1 balances precision and recall, and ROC AUC measures ranking discrimination across classification thresholds.

12.1 Classification Evaluation

Metric	Result
Accuracy	[Run supplied Python code]
Precision	[Run supplied Python code]
Recall	[Run supplied Python code]
F1 Score	[Run supplied Python code]
ROC AUC	[Run supplied Python code]

Table 6. Classification Model Performance

Confusion matrix:

	Predicted No	Predicted Yes
--	--------------	---------------

Actual No	True Negative: [Run code]	False Positive: [Run code]
Actual Yes	False Negative: [Run code]	True Positive: [Run code]

Table 7. Confusion Matrix

12.2 Classification Interpretation

Because the project uses class balancing, recall and F1-score should be considered alongside accuracy. For a retention use case, false negatives are important because they represent churners that the model failed to identify. False positives also matter because they can lead to unnecessary retention outreach.

12.3 Overfitting and Underfitting

The supplied code reports test-set metrics but does not calculate a separate training-set metric table or cross-validation result. Therefore, a definitive claim about overfitting or underfitting cannot be made from the code alone. A stronger final evaluation would compare training and test performance or use cross-validation.

13. RESULTS AND FINDINGS

27. Churn is a minority outcome in the dataset, making class-aware evaluation important.
28. Contract type is a key observed churn-related variable; the project analysis identifies month-to-month contracts as having substantially higher churn than longer-term contracts.
29. Monthly charges show a visible association with churn status in the supplied EDA workflow, with the generated report noting higher monthly charges among customers who leave.
30. The chi-square analysis is designed to test whether contract type and churn are statistically associated; the exact statistic and p-value should be inserted from the executed code.
31. Logistic Regression provides a reproducible binary churn classifier, and its coefficients can be inspected to identify influential encoded features.

The final numerical model findings should be populated from the generated `model_metrics.csv` and `confusion_matrix.csv` files produced by the supplied Python program.

14. DISCUSSION

The project combines descriptive analysis, statistical testing, and supervised machine learning rather than relying on a prediction score alone. The exploratory results direct attention to contract structure and monthly charges, while the chi-square test provides a formal test of association for contract type. Logistic Regression then combines multiple customer attributes to produce a binary prediction.

From a business perspective, the workflow can be used as a screening mechanism for retention planning. However, the model identifies statistical patterns in historical data; it does not establish that changing a particular customer attribute will cause churn to decrease. Any retention action should therefore be evaluated through appropriate business experiments and monitoring.

15. LIMITATIONS

- The supplied Python script does not report the original data-source URL, licensing details, or access date.
- Duplicate records are not explicitly checked or removed in the submitted workflow.
- Outliers are visualized but not formally treated, and no robust outlier-sensitivity analysis is included.
- The project uses a single 80:20 train-test split and does not include cross-validation in the supplied implementation.

- Training-versus-test performance is not compared, so overfitting cannot be assessed rigorously from the current output alone.
- Only the variables present in the supplied dataset are available; external economic, competitive, service-quality, and customer-interaction variables are not included.

16. CONCLUSION

This capstone project demonstrates an end-to-end customer churn prediction workflow for a telecommunications dataset. The workflow covers data cleaning, preprocessing, exploratory analysis, statistical testing, Logistic Regression model development, and classification evaluation. The analysis focuses particularly on contract type and monthly charges and provides a structured method for identifying customers who may require further retention attention. The supplied code is reproducible and generates model metrics, a confusion matrix, and feature coefficients for final interpretation. The project therefore establishes a practical analytical foundation for telecom churn monitoring, while recognizing that deployment and business intervention would require further validation.

17. RECOMMENDATIONS AND FUTURE WORK

17.1 Recommendations

- Use churn-risk predictions as a prioritization signal for customer-retention teams rather than as an automatic decision rule.
- Review month-to-month customers for retention opportunities because the supplied analysis identifies this contract group as having substantially higher churn.
- Examine customers with comparatively high monthly charges as a potential retention segment, while validating the reason for the observed association before changing pricing.
- Track recall and false-negative counts in addition to accuracy when the business objective is to identify as many potential churners as possible.
- Use the model coefficients and segment-level analysis to design targeted retention campaigns, followed by controlled measurement of campaign outcomes.

17.2 Future Work

- Add cross-validation and hyperparameter tuning to obtain a more robust estimate of model performance.
- Compare Logistic Regression with other classification methods such as decision trees, random forests, or gradient-boosting models.
- Add calibration analysis so predicted probabilities can be interpreted as usable risk scores.
- Investigate customer lifetime value, service quality, complaints, usage patterns, and interaction history if such data become available.
- Develop a deployment pipeline or dashboard for periodic scoring and monitoring of churn-risk segments.

18. REFERENCES

The following references correspond to technologies and methodological components explicitly used in the supplied implementation. The exact dataset source should be inserted from the source used by the student.

Scikit-learn. (n.d.). LogisticRegression documentation.

Scikit-learn. (n.d.). ColumnTransformer and Pipeline documentation.

Scikit-learn. (n.d.). OneHotEncoder, StandardScaler, SimpleImputer, and train_test_split documentation.

Scikit-learn. (n.d.). Classification metrics documentation: accuracy, precision, recall, F1-score, ROC AUC, and confusion matrix.

SciPy. (n.d.). `scipy.stats.chi2_contingency` documentation.

Pandas. (n.d.). `pandas.to_numeric` and `DataFrame` documentation.

Matplotlib. (n.d.). Matplotlib documentation.

Seaborn. (n.d.). Seaborn documentation.

Telco Customer Churn dataset. Insert the exact dataset provider, title, URL, access date, and license used for this project.

APPENDIX A. CODE

The complete Python implementation was supplied separately as “final_capstone_telco_churn.py”. The following subsections summarize the reproducibility structure. The full source file should be attached or included with the final submission.

A.1 Data Loading and Cleaning

```
df = pd.read_csv(path)
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['Churn'] = df['Churn'].str.strip()
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())
df['Churn_Code'] = df['Churn'].map({'Yes': 1, 'No': 0})
```

A.2 Preprocessing

```
preprocessor = ColumnTransformer([
    ('num', Pipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('scaler', StandardScaler())
    ]), num_cols),
    ('cat', Pipeline([
        ('imputer', SimpleImputer(strategy='most_frequent')),
        ('onehot', OneHotEncoder(handle_unknown='ignore'))
    ]), cat_cols)
])
```

A.3 Train-Test Split and Model

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
model = LogisticRegression(
    max_iter=5000, class_weight='balanced', random_state=42
)
```

A.4 Evaluation

```
y_pred = pipeline.predict(X_test)
y_prob = pipeline.predict_proba(X_test)[:, 1]
accuracy_score(y_test, y_pred)
precision_score(y_test, y_pred)
recall_score(y_test, y_pred)
f1_score(y_test, y_pred)
roc_auc_score(y_test, y_prob)
confusion_matrix(y_test, y_pred)
```

A.5 Statistical Analysis

```
contingency = pd.crosstab(df['Contract'], df['Churn'])
chi2, p_value, _, _ = chi2_contingency(contingency)
```

APPENDIX B. ADDITIONAL FIGURES AND TABLES

Insert the generated project outputs here after executing the supplied Python file.

Figure 4. Customer Churn Distribution — generated file: churn_distribution.png

[INSERT IMAGE HERE]

Figure 5. Churn by Contract Type — generated file: contract_churn.png

[INSERT IMAGE HERE]

Figure 6. Monthly Charges by Churn Status — generated file: monthly_charges_churn.png

[INSERT IMAGE HERE]

Additional model output files generated by the supplied code:

- model_metrics.csv — Accuracy, Precision, Recall, F1 Score, and ROC AUC.
- confusion_matrix.csv — True Negative, False Positive, False Negative, and True Positive counts.
- classification_report.csv — class-level precision, recall, F1-score, and support.